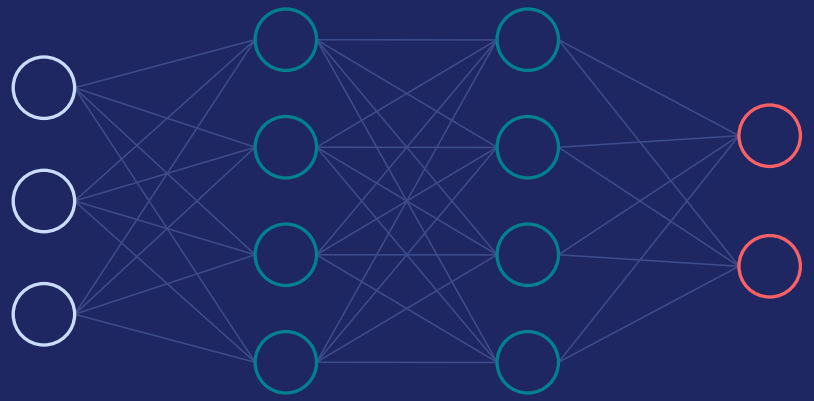
기계학습특론



CHAPTER 6-2

다층 퍼셉트론의 학습

데이터 · 손실 함수 · 경사 하강법 · 오차 역전파 · 최적화와 정규화 · 성능 평가 지표 · PyTorch 실습

성명 : 최승호
이메일 : jcn99250@naver.com

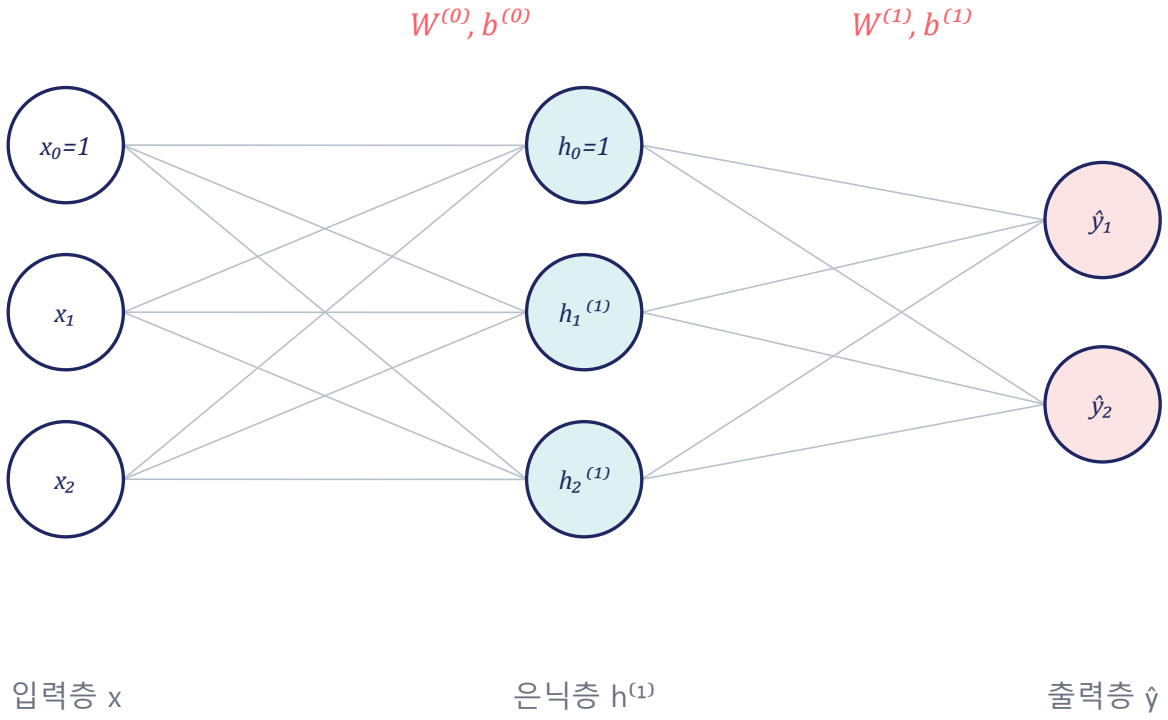
2026-2학기 · PyTorch 2.x 기준 개정판

학습 목표

1. MLP 학습을 «손실 함수의 최소화» 문제로 정식화하고 데이터·모델·최적화의 역할을 구분한다
2. 회귀(MSE)·이진 분류(BCE)·다중 분류(CCE) 손실 함수를 유도하고 수치 예제로 계산한다
3. 경사 하강법과 미니배치 SGD, 학습률의 의미를 이해한다
4. 연쇄법칙으로 오차 역전파 알고리즘을 유도하고 PyTorch autograd 와 대응시킨다
5. 기울기 소실·폭주, 모멘텀·Adam·AdamW, 학습률 스케줄, 정규화·드롭아웃 등 현대 학습 기법을 익힌다
6. 평가 지표(정확도·혼동행렬·정밀도·재현율·F1·R^2)로 모델을 검증하고 MNIST 실습 결과를 해석한다

다층 퍼셉트론 복습 — 순전파

6-1 강의: 층 하나 = 선형 변환 + 비선형 활성화. 학습의 목표는 이 식 안의 W, b 를 정하는 것



$$\hat{\mathbf{y}} = f_2(\mathbf{W}^{(1)} f_1(\mathbf{W}^{(0)}\mathbf{x} + \mathbf{b}^{(0)}) + \mathbf{b}^{(1)})$$

- 학습 데이터가 주어지면 순전파로 $\hat{y}$를 계산할 수 있다 — 단, $W^{(0)}$, $b^{(0)}$, $W^{(1)}$, $b^{(1)}$ 값이 있어야 한다
- **«학습» = 정답 y 와 예측 $\hat{y}$ 의 차이가 작아지도록 파라미터를 찾는 과정**
  - 무엇을 «차이»로 잴 것인가 → 손실 함수
  - 어떻게 파라미터를 바꿀 것인가 → 경사 하강법
  - 기울기를 어떻게 효율적으로 계산할 것인가 → 오차 역전파
  - 잘 배웠는지 어떻게 확인할 것인가 → 평가 지표와 검증 데이터

2-2-2 예시에서 파라미터는 $W^{(0)}(2\times2)+b^{(0)}(2)+W^{(1)}(2\times2)+b^{(1)}(2)$ = 12개.
MNIST MLP(784-600-500-10)는 776,510개

학습 = 경험적 위험 최소화 (Empirical Risk Minimization)

훈련 데이터에서의 평균 손실을 최소화하는 파라미터 W* 를 찾는다

$$\mathbf{W}^* = \arg\min_{\mathbf{W}} L(\mathbf{W}) = \arg\min_{\mathbf{W}} \frac{1}{N}\sum_{i=1}^{N} \ell\left(y_i,\ \hat{y}_i(\mathbf{x}_i; \mathbf{W})\right)$$

① 데이터 $\mathcal{D}$

- 관측 (x_i, y_i) N개
- 훈련 / 검증 / 테스트로 분할
- 표준화·증강 등 전처리

② 모델 + 손실 함수 ℓ

- $\hat{y}$ = MLP(x; W)
- $\ell(y, \hat{y})$: MSE / BCE / CCE
- 미분 가능해야 한다

③ 최적화 알고리즘

- 경사 하강법 $W \leftarrow W - \eta \nabla L$
- 기울기는 역전파로 계산
- SGD·모멘텀·Adam·AdamW

- 훈련 손실이 작아도 새 데이터에서 잘 맞는다는 보장은 없다 (일반화) → 검증 데이터로 감시, 정규화·조기 종료로 제어 (슬라이드 24)
- MLP 의 L(W) 는 비볼록(non-convex) → 전역 최적해 보장 없음. 그러나 실제로는 «충분히 좋은» 국소해에 잘 도달 (초기화·최적화기가 중요)

다층 퍼셉트론 학습 1 — 학습 데이터

어떤 문제를 해결하기 위해 제공되는 관측값 x_i 와, 모델이 예측해야 하는 실제 값 y_i (정답, label, ground truth)

$$\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{N}, \qquad \mathbf{x}_i \in \mathbb{R}^d, \quad y_i \in \mathbb{R} \text{ or } \{0, 1\} \text{ or } \{1, \ldots, C\}$$

예시	x_i (특징 벡터)	y_i (정답)	문제 유형	출력층 / 손실
주가 예측	PMI, VIX, 코스피 지수 (300일치 관측) → $x_i \in \mathbb{R}^3$	다음 날 코스피 $\in \mathbb{R}$	회귀 (Regression)	항등 / MSE
고양이·개 사진	600×800 픽셀 → $x_i \in \mathbb{R}^{480000}$ (10,000장)	0 (고양이) 또는 1 (개)	이진 분류	시그모이드 / BCE
Fashion-MNIST	28×28 픽셀 → $x_i \in \mathbb{R}^{784}$ (60,000장)	10개 클래스 중 하나 (원-핫)	다중 클래스 분류	소프트맥스 / CCE

- **y_i 의 형태가 «출력층 활성화 함수»와 «손실 함수»를 결정한다 (다음 슬라이드부터)**
- 데이터 수집·정제·레이블링이 실제 프로젝트 시간의 대부분 — 레이블 품질이 모델 상한을 정한다
- 실습 2: torchvision.datasets.MNIST 로 로드 → transforms 로 텐서화·표준화 → DataLoader 로 미니배치 구성

데이터 분할과 전처리 — 학습 전에 정해야 할 것

훈련 데이터로 파라미터를, 검증 데이터로 하이퍼파라미터를, 테스트 데이터로 최종 성능을 정한다



MNIST 실습 2의 분할 예 (random_split, 시드 고정)

```
transform = transforms.Compose([transforms.ToTensor(),
# [0,255] → [0,1]

transforms.Normalize((0.1307,), (0.3081,))])  # (x-μ)/σ
```

훈련 데이터

- 파라미터 W 학습에 사용
- 매 에폭 섞어서(shuffle) 미니배치 구성
- 손실이 계속 내려가는 것이 정상

검증 데이터

- 학습률·층 수·드롭아웃 등 하이퍼파라미터 선택
- 과적합 감시 → 조기 종료 시점 결정
- 여러 번 들여다보므로 «약간 오염»됨

테스트 데이터

- 최종 보고용, 단 한 번만 평가
- 테스트로 모델을 고르면 성능이 과대평가(테스트 누수)
- 논문·경진대회의 «리더보드»가 이 역할

표준화: 입력 분포를 평균 0·분산 1로 맞추면 가중합 크기가 안정되어 학습률 선택이 쉬워진다. 통계량은 «훈련 데이터»로만 계산

다층 퍼셉트론 학습 2 — 손실 함수 (Loss Function)

데이터를 통해 산출한 모델의 예측값과 실제값의 차이를 «하나의 숫자»로 표현하는 지표

y_i 가 연속형 (회귀)

- 평균 제곱 오차 MSE
- 출력층: 항등 함수 (선형)
- 예: 주가, 온도, 나이 예측

$y_i \in \{0, 1\}$ (이진 분류)

- 이진 교차 엔트로피 BCE
- 출력층: 시그모이드 → 확률
- 예: 스팸 여부, 질병 유무

$y_i \in \{1,\ldots,C\}$ (다중 분류)

- 범주형 교차 엔트로피 CCE
- 출력층: 소프트맥스 → 확률 벡터
- 예: 숫자 0~9, 사물 1000종

- **좋은 손실 함수의 조건**
  - 예측이 정답에 가까울수록 작고, 완전히 맞으면 0 (또는 최소)
  - 파라미터에 대해 미분 가능 → 경사 하강법 적용 가능 (정확도·F1 같은 지표는 미분 불가 → 직접 최적화 못 함)
  - 확률적 해석: MSE = 가우시안 잡음 가정의 최대가능도, 교차 엔트로피 = 베르누이/범주 분포의 최대가능도 (음의 로그가능도, NLL)
- **손실(loss)은 학습을 «이끄는» 목적 함수, 평가 지표(metric)는 결과를 «보고하는» 값 — 둘은 다를 수 있다**

손실 함수 ① 회귀 — 평균 제곱 오차 (MSE)

y_i 가 연속형 값일 때, 예측 $\hat{y}_i$ 와의 차이를 제곱해 평균

$$\mathrm{MSE} = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$$

- **표기**
  - y_i : i번째 샘플의 정답, $\hat{y}_i = w^T x_i$ (선형 회귀) 또는 $\mathrm{MLP}(x_i)$ 의 출력
  - 제곱 → 부호 제거, 큰 오차에 큰 벌점, 미분이 간단 ($\partial/\partial\hat{y}_i = -2(y_i - \hat{y}_i)/N$)
- **변형**
  - MAE (평균 절대 오차): 이상치(outlier)에 덜 민감하지만 0에서 미분 불연속
  - Huber / Smooth-L1: 작은 오차는 제곱, 큰 오차는 절대값 — 둘의 절충 (PyTorch nn.HuberLoss)
- PyTorch: nn.MSELoss(), nn.L1Loss(), nn.HuberLoss(delta=1.0)

i	y_i	$\hat{y}_i$	$(y_i - \hat{y}_i)^2$
1	2.0	2.5	0.25
2	3.0	2.0	1.00
3	5.0	5.5	0.25
4	1.0	1.2	0.04
		MSE	(0.25+1+0.25+0.04)/4 = 0.385

MSE 의 단위는 y 의 제곱 → 보고할 때는 RMSE = √MSE 또는 R^2 (슬라이드 26) 를 함께 제시

주의

출력층에 시그모이드를 두고 MSE 를 쓰면 시그모이드 포화 영역에서 기울기가 0에 가까워져 학습이 매우 느려진다 → 분류에는 교차 엔트로피를 쓴다

손실 함수 ② 이진 분류 — 이진 교차 엔트로피 (BCE)

$y_i \in \{0, 1\}$, 모델 출력 $\hat{y}_i = \sigma(z_i)$ 는 «$y_i = 1$ 일 확률»

$$\mathrm{BCE} = -\frac{1}{N}\sum_{i=1}^{N}\left[y_i \ln \hat{y}_i + (1-y_i)\ln(1-\hat{y}_i)\right], \qquad \hat{y}_i = \sigma(z_i)$$

직관

$y_i = 1$ 이면 $-\ln \hat{y}_i$ 만 남음 → $\hat{y}_i \to 1$ 일 때 0, $\hat{y}_i \to 0$ 일 때 ∞. $y_i = 0$ 이면 $-\ln(1-\hat{y}_i)$ → 반대. 즉 «정답 쪽 확률의 음의 로그»

유도

베르누이 가능도 $\prod \hat{y}_i^{y_i}(1-\hat{y}_i)^{1-y_i}$ 의 음의 로그 → 최대가능도 추정 = BCE 최소화

y_i	$\hat{y}_i$	손실 $-[y \ln \hat{y} + (1-y)\ln(1-\hat{y})]$
1	0.9	$-\ln 0.9 = 0.105$
1	0.5	$-\ln 0.5 = 0.693$
1	0.1	$-\ln 0.1 = 2.303$
0	0.1	$-\ln 0.9 = 0.105$

- 시그모이드 + BCE 의 결합 미분은 $\partial L/\partial z_i = (\hat{y}_i - y_i)/N$ 으로 매우 단순 (실습 1 §2)
- PyTorch: nn.BCEWithLogitsLoss() — 로짓 z 를 입력받아 log-sum-exp 로 안정 계산. nn.BCELoss() 에 확률을 넣는 방식은 ln 0 위험
- 클래스 불균형: pos_weight 인자로 양성 클래스 가중, 또는 Focal loss (쉬운 샘플의 손실을 줄임)

손실 함수 ③ 다중 클래스 분류 — 범주형 교차 엔트로피 (CCE)

y_i 는 원-핫 벡터, $\hat{y}_i = \text{softmax}(z_i)$ 는 C개 클래스 확률 벡터

$$\text{CCE} = -\frac{1}{N}\sum_{i=1}^{N}\sum_{k=1}^{C} y_{ik}\ln \hat{y}_{ik}, \qquad \hat{\mathbf{y}}_i = \text{softmax}(\mathbf{z}_i)$$

$$\hat{\mathbf{y}} = [0.02,\ 0.90,\ 0.05,\ 0.03]^\top,\ y = \text{class } 2 \Rightarrow L = -\ln 0.90 \approx 0.105$$

k	z_k (로짓)	ŷ_k = softmax	y_k (정답=2)	−y_k ln ŷ_k
1	0.2	0.02	0	0
2	4.0	0.90	1	0.105
3	1.1	0.05	0	0
4	0.6	0.03	0	0
			합 =	0.105

- 원-핫 y_i 는 정답 위치만 1 → 이중 합에서 «정답 클래스의 −ln ŷ_{i,y}» 만 남는다 (BCE 의 자연스러운 확장)
- 소프트맥스 + CCE 의 결합 미분: $\partial L/\partial z_i = (\hat{y}_i - y_i)/N$ — 이진 경우와 같은 형태
- **PyTorch: nn.CrossEntropyLoss() = LogSoftmax + NLLLoss. 정수 레이블(0~C−1)을 그대로 받으며, 모델은 로짓 z만 출력**
- label_smoothing=0.1: 정답 확률 목표를 1 대신 0.9 로 완화 → 과신 방지, 일반화 향상 (Inception-v3 2016; ViT·LLM 훈련에서 흔히 사용)

혼동하기 쉬운 점

- CCE(원-핫) ≡ Sparse CCE(정수 레이블) — 표현만 다름
- 모델 끝에 Softmax 층을 추가하면 이중 소프트맥스 → 기울기 약화
- 이진 분류 = C=2 인 CCE = BCE (수학적으로 동치)

참고: 손실 함수 연습 문제

모델에 «6번째 샘플»이 입력될 때, 다섯 클래스 분류 문제의 손실을 계산해 보자 (원본 강의 연습문제 개정)

① 출력층 가중합 $z = [2.0, 0.5, 1.0, 0.1, -1.0]^T$

② 소프트맥스 $\hat{y} = [0.56, 0.12, 0.21, 0.08, 0.03]^T$ (합 = 1.0, 반올림)

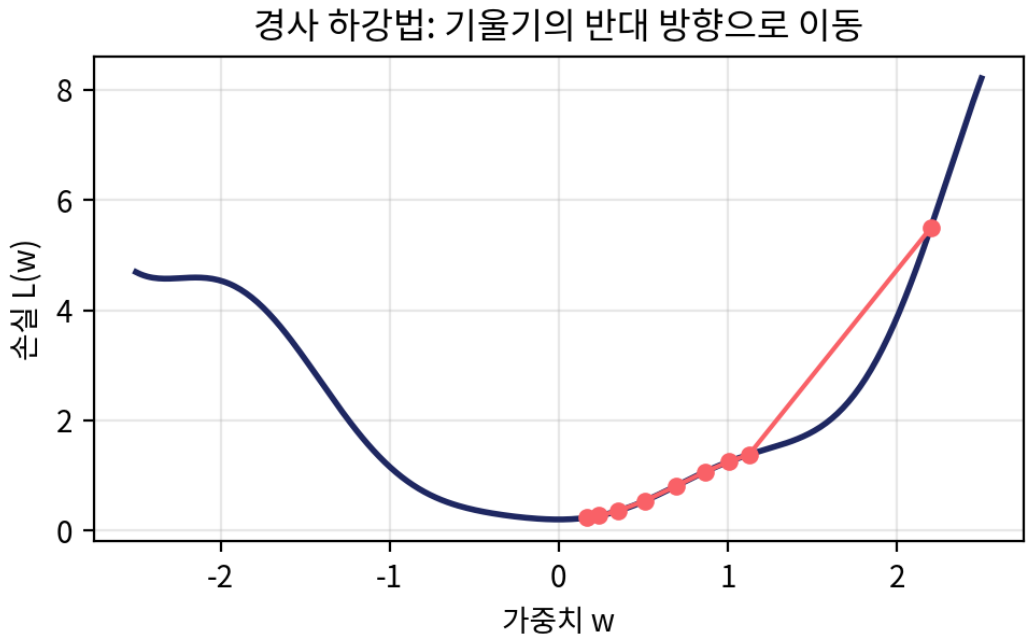
③ 정답 (원-핫) $y = [0, 0, 1, 0, 0]^T$ (정답 = 클래스 3)

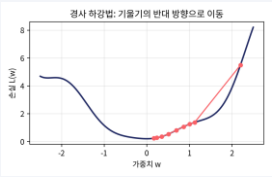
④ CCE 손실 $L = -\Sigma_k y_k \ln \hat{y}_k = -\ln 0.21 \approx 1.56$

- **풀이 확인**
  - e^z = [7.39, 1.65, 2.72, 1.11, 0.37], 합 = 13.23 → ŷ = e^z / 13.23
  - 예측 클래스 argmax ŷ = 1 ≠ 정답 3 → 오답. 손실 1.56 은 «정답 확률 21%» 를 의미
- **생각해 볼 문제**
  - z_3 을 1.0 → 3.0 으로 올리면 손실은? ($\hat{y}_3 \approx 0.66 \rightarrow L \approx 0.42$)
  - 정답이 클래스 1 이었다면 손실은? ($-\ln 0.56 \approx 0.58$)
  - 무작위 추측(모든 $\hat{y}_k = 0.2$)의 손실 $-\ln 0.2 \approx 1.61$ 과 비교해 보자 — 학습 초기 손실의 기준선

다층 퍼셉트론 학습 3 — 경사 하강법 (Gradient Descent)

손실 L(W) 가 가장 가파르게 «감소»하는 방향 = 기울기의 반대 방향으로 파라미터를 조금씩 이동





- **학습률 η (learning rate)**
  - 너무 작으면 수렴이 느리고, 너무 크면 발산·진동. 가장 중요한 하이퍼파라미터 (보통 10^{-4} ~ 10^{-1}, 로그 스케일로 탐색)
  - 고정하지 않고 스케줄로 바꾸는 것이 표준 (슬라이드 22)
- **종료 조건**
  - 정해진 에폭 수, 검증 손실 정체(early stopping), 기울기 크기 < 임계값
- **필요한 것: ∂L/∂W — 파라미터가 77만 개면 편미분도 77만 개 → 효율적 계산법이 «오차 역전파»**

1차 정보(기울기)만 사용: 뉴턴법 같은 2차 방법은 헤시안(77만×77만)이 너무 커서 실용적이지 않다

미니배치 확률적 경사 하강법 (Mini-batch SGD)

전체 N개 대신 무작위로 뽑은 B개 샘플로 기울기를 «추정» — 계산량과 잡음의 절충

$$\mathbf{W}_{t+1} = \mathbf{W}_t - \alpha \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \nabla_{\mathbf{W}} \ell(\mathbf{x}_i, y_i; \mathbf{W}_t)$$

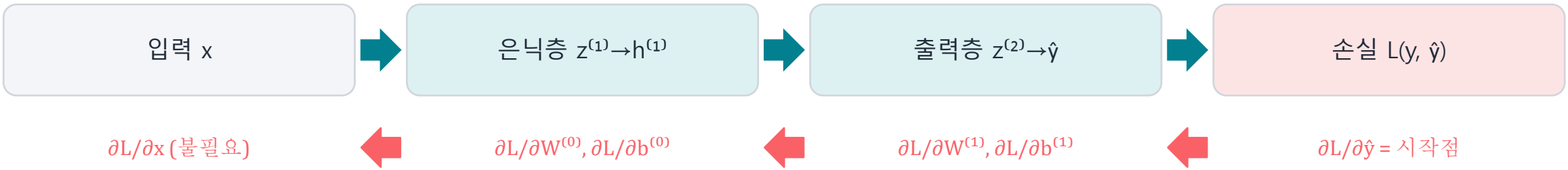
방식	배치 크기	기울기 1회 계산 비용	특징
배치 GD	N (전체)	매우 큼 (N=54,000 통과)	정확한 기울기, 메모리 부담, 국소 최소에 갇히기 쉬움
SGD (순수)	1	매우 작음	잡음 큼 → 탈출 효과, GPU 병렬성 활용 못 함
미니배치 SGD	32 ~ 1024	중간 (GPU 행렬곱에 최적)	표준. 잡음이 «암묵적 정규화» 역할 (일반화에 유리)

- 용어: 1 iteration(step) = 미니배치 1개로 갱신 1회, 1 epoch = 훈련 데이터 전체를 한 번 통과 (MNIST 훈련 54,000, B=128 → 422 step)
- 매 에폭 데이터를 섞어(shuffle) 배치 구성이 달라지게 한다 → DataLoader(shuffle=True)
- 배치 크기를 키우면 학습률도 함께 키운다 (선형 스케일링 규칙, Goyal et al. 2017); 대형 모델은 수천~수백만 토큰 배치 + warm-up
- 모멘텀·Adam 은 이 «잡음 섞인 기울기»를 평활화해 안정적으로 만든다 (슬라이드 20~21)

오차 역전파 (Backpropagation) — 정의

예측값과 실제값의 차이(손실)를 최소화하기 위해 출력층에서 입력층 방향으로 모든 가중치의 기울기를 계산하는 알고리즘

순전파 (forward): 예측값 계산



역전파 (backward): 출력층부터 거꾸로 기울기 계산 · 갱신

핵심 아이디어

- 연쇄법칙(chain rule)을 층 단위로 적용
- 뒤층에서 계산한 «오차 신호» δ 를 앞층으로 전달해 재사용
- 각 층은 자기 «국소 기울기»만 알면 된다

계산량

- 순전파 1회 비용의 약 2배로 «모든» 파라미터의 편미분을 얻음
- 수치 미분(파라미터마다 순전파 반복)은 77만 배 느림
- 역방향 자동 미분(reverse-mode AD)의 특수한 경우

역사

- Werbos (1974), Rumelhart·Hinton·Williams (1986)로 정착
- PyTorch autograd, JAX, TensorFlow 가 자동 수행
- 실습 1 §2: 손 계산 vs loss.backward() 일치 확인

미분 연쇄법칙과 역전파

합성 함수의 미분 = 각 단계 미분의 곱. 2층 MLP 에서 $W^{(0)}$ 에 대한 손실의 기울기:

$$\frac{\partial L}{\partial \mathbf{W}^{(0)}} = \frac{\partial L}{\partial \hat{\mathbf{y}}} \cdot \frac{\partial \hat{\mathbf{y}}}{\partial \mathbf{z}^{(2)}} \cdot \frac{\partial \mathbf{z}^{(2)}}{\partial \mathbf{h}^{(1)}} \cdot \frac{\partial \mathbf{h}^{(1)}}{\partial \mathbf{z}^{(1)}} \cdot \frac{\partial \mathbf{z}^{(1)}}{\partial \mathbf{W}^{(0)}}$$

인수	의미	2-2-1 예 (시그모이드 + BCE)
$\partial L/\partial \hat{y} \cdot \partial \hat{y}/\partial z^{(2)}$	손실 → 출력층 가중합 (출력층 오차 $\delta^{(2)}$)	$(\hat{y} - y)/N$ ← 시그모이드+BCE 결합 미분
$\partial z^{(2)}/\partial h^{(1)}$	출력층 가중합 → 은닉층 출력	$W^{(1)T}$ (가중치 그대로)
$\partial h^{(1)}/\partial z^{(1)}$	은닉층 활성화 함수의 도함수	$\sigma'(z^{(1)}) = h^{(1)} \odot (1 - h^{(1)})$
$\partial z^{(1)}/\partial W^{(0)}$	은닉층 가중합 → 가중치	x^T (입력 그대로)

- 앞의 세 인수의 곱이 은닉층 오차 $\delta^{(1)}$ — 이것을 한 번 계산해 두면 $W^{(0)}$ 와 $b^{(0)}$ 의 기울기에 모두 재사용
- 층이 더 깊어져도 «$\delta^{(k)} = (W^{(k)T}\ \delta^{(k+1)}) \odot f'(z^{(k)})$» 를 반복하면 된다 → 다음 슬라이드의 알고리즘
- ReLU 의 도함수는 0 또는 1 → 활성 노드에서는 오차가 감쇠 없이 그대로 통과 (6-1 강의 «기울기 소실»)

오차 역전파 알고리즘 — 일반식

출력층 오차 $\delta^{(L)}$ 에서 시작해 δ 를 한 층씩 뒤로 전파하고, 각 층의 기울기는 «오차 × 입력^{T}»

$$\delta^{(L)} = \hat{\mathbf{y}} - \mathbf{y}, \qquad \delta^{(k)} = (\mathbf{W}^{(k)\top} \delta^{(k+1)}) \odot f_k'(\mathbf{z}^{(k)}), \qquad \frac{\partial L}{\partial \mathbf{W}^{(k-1)}} = \delta^{(k)} \mathbf{h}^{(k-1)\top}$$

```
# 순전파: h[0] = x
for k in range(L):
    z[k+1] = W[k] @ h[k] + b[k]
    h[k+1] = f(z[k+1])            # 마지막 층은 softmax/sigmoid/identity
loss = ℓ(y, h[L])

# 역전파: 출력층 오차 (softmax+CCE 또는 sigmoid+BCE 이면 단순히 ŷ - y)
delta = (h[L] - y) / N
for k in reversed(range(L)):
    dW[k] = delta @ h[k].T        # ∂L/∂W(k) = δ(k+1) h(k)ᵀ
    db[k] = delta.sum(axis=1)
    if k > 0:
        delta = (W[k].T @ delta) * f_prime(z[k])   # 앞 층으로 전파

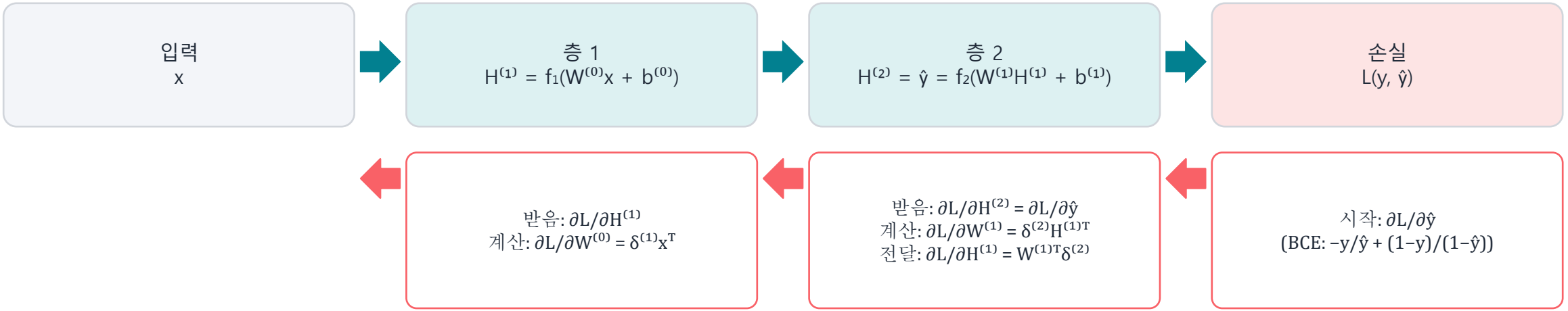
# 갱신
for k in range(L): W[k] -= lr * dW[k]; b[k] -= lr * db[k]
```

- 필요한 저장값: 순전파의 $z^{(k)}$, $h^{(k)}$ (→ 메모리 사용량이 « 깊이 × 배치»에 비례하는 이유)
- 행렬 연산만으로 구성 → GPU 에서 순전파와 같은 속도로 실행
- 실습 1 §2 의 backward() 함수가 이와 같은 알고리즘 (L = 2, f = 시그모이드; 코드는 X @ W 행 벡터 규약이라 전치형)
- PyTorch 에서는 loss.backward() 한 줄 — 각 연산의 국소 미분을 계산 그래프에 기록해 두었다가 거꾸로 곱한다

역전파 흐름 — 국소 기울기(local gradient)의 연결

각 층은 «출력에 대한 손실의 기울기»를 받아 «자기 입력과 가중치에 대한 기울기»를 계산해 앞으로 넘긴다

$$\frac{\partial L}{\partial \mathbf{W}^{(0)}} = \frac{\partial \mathbf{H}^{(1)}}{\partial \mathbf{W}^{(0)}} \cdot \frac{\partial L}{\partial \mathbf{H}^{(1)}}, \qquad \frac{\partial L}{\partial \mathbf{H}^{(1)}} = \frac{\partial \mathbf{H}^{(2)}}{\partial \mathbf{H}^{(1)}} \cdot \frac{\partial L}{\partial \mathbf{H}^{(2)}}$$



- «Error < 허용치?» 를 만족할 때까지 순전파 → 손실 → 역전파 → 갱신을 반복 (에폭 루프). 실습 2 의 train() 함수가 이 순환
- 층 종류(선형·활성화·정규화·어텐션)가 달라도 인터페이스는 같다: 받은 기울기 × 국소 기울기 → 앞으로 전달. 이것이 «모듈식» 딥러닝 프레임워크의 설계 원리

PyTorch autograd — 손 계산과 자동 미분의 일치 확인 (실습 1 §2)

동일한 가중치로 NumPy 역전파와 torch.autograd 를 실행하면 기울기가 1e-8 이내로 일치한다

```
# (1) NumPy: 손으로 구현한 역전파
d2  = (Y_hat - Y) / N                  # ∂L/∂Z2
dW1 = H1.T @ d2                        # ∂L/∂W1 = H1ᵀ δ2
d1  = (d2 @ W1.T) * dsigmoid(Z1)       # ∂L/∂Z1 = (δ2 W1ᵀ) ⊙ σ'(Z1)
dW0 = X.T @ d1                         # ∂L/∂W0 = Xᵀ δ1

# (2) PyTorch: 자동 미분
tW0, tb0, tW1, tb1 = [torch.tensor(a, dtype=torch.float32, requires_grad=True) for a in (W0,
b0, W1, b1)]
H   = torch.sigmoid(Xt @ tW0 + tb0)
Yp  = torch.sigmoid(H @ tW1 + tb1)
loss = F.binary_cross_entropy(Yp, Yt)
loss.backward()                        # ← 역전파 자동 수행
print(np.abs(dW0 - tW0.grad.numpy()).max())   # 3.8e-09 ✓
```

기울기	최대 |손 계산 − autograd|
$dW^{(0)}$	3.84×10^{-9}
$db^{(0)}$	3.24×10^{-9}
$dW^{(1)}$	7.98×10^{-9}
$db^{(1)}$	1.44×10^{-8}

- **autograd 동작 원리**
  - requires_grad=True 텐서에 가해진 모든 연산을 «계산 그래프»에 기록 (동적 그래프)
  - backward() 는 그래프를 역순으로 돌며 각 연산의 국소 미분(벡터-야코비안 곱)을 누적
  - 실무에서는 nn.Module 파라미터가 자동으로 requires_grad=True; optimizer.zero_grad() 로 누적 기울기를 매 스텝 비움

차이는 float32 반올림 수준 — 손으로 유도한 역전파가 정확함을 «실행으로» 증명

기울기 소실과 폭주 (Vanishing / Exploding Gradients)

$\delta^{(k)} = (W^{(k)T}\,\delta^{(k+1)}) \odot f'(z^{(k)})$ 를 L번 반복하면 «W 의 크기 × f' 의 크기»가 L제곱으로 누적된다

$$\left\| \frac{\partial L}{\partial \mathbf{W}^{(0)}} \right\| \lesssim \prod_{k=1}^{L} \|\mathbf{W}^{(k)}\| \, \|f_k'\| \;\to\; 0 \quad (\sigma' \le 0.25)$$

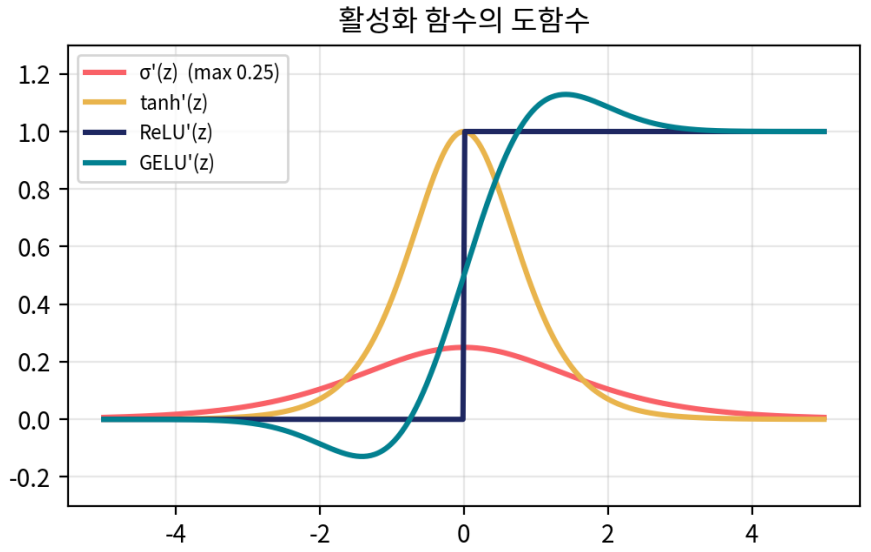
소실 (vanishing)

- 시그모이드 σ' ≤ 0.25 → 10층이면 10^{-6} 배
- 앞쪽 층 가중치가 거의 갱신되지 않음
- → 깊은 망 학습 실패 (1990년대)

폭주 (exploding)

- ‖W‖ > 1 이 누적 → 기울기 ∞, 손실 NaN
- RNN·매우 깊은 망에서 흔함
- → 기울기 클리핑으로 상한 제한

해결책	연도	원리
ReLU 계열 활성화	2011	f' ∈ {0, 1} → 감쇠 없음
Xavier / He 초기화	2010 / 2015	‖W‖ 의 기대값을 1 근처로
배치·레이어 정규화	2015 / 2016	각 층 입력 분포를 재조정
잔차 연결	2015	항등 경로로 기울기 직접 전달
기울기 클리핑	2013	‖∇‖ > c 이면 c 로 축소



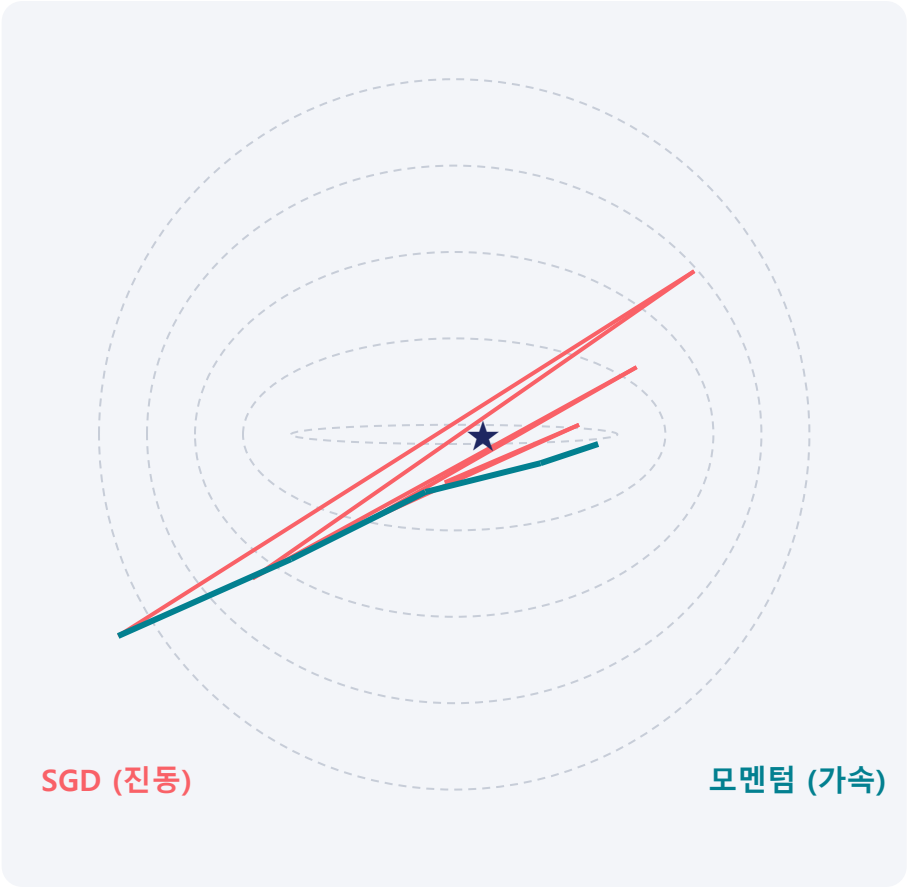
- 2010년 이전: «3층 이상은 학습 안 된다» → 위 다섯 기법의 조합으로 2015년 152층(ResNet), 현재 수백 층 트랜스포머까지
- 실습 2 §6 실험 A(시그모이드+SGD) 91% vs D(GELU+BN+AdamW) 98% — 같은 구조·같은 에폭에서의 차이

현대 최적화 ① 모멘텀 (Momentum)

기울기를 «속도»에 누적해 진동을 줄이고 골짜기 방향으로 가속 — 관성을 가진 공

$$\mathbf{v}_t = \beta\,\mathbf{v}_{t-1} + \nabla L(\mathbf{W}_t), \qquad \mathbf{W}_{t+1} = \mathbf{W}_t - \alpha\,\mathbf{v}_t$$

- β ≈ 0.9: 최근 ~10 스텝 기울기의 지수 이동 평균 방향으로 이동
- 좁고 긴 골짜기에서 SGD 는 벽 사이를 진동하지만, 모멘텀은 진동 성분이 상쇄되고 골짜기 방향 성분은 누적
- Nesterov 모멘텀: «먼저 관성 방향으로 가 본 뒤» 그 지점의 기울기 사용 — 약간 더 안정 (torch.optim.SGD(nesterov=True))
- 실습 1: SGD(lr=0.5, momentum=0.9) 로 XOR 3000 epoch 수렴



PyTorch: torch.optim.SGD(params, lr=0.1, momentum=0.9, nesterov=True, weight_decay=5e-4) — 이미지 분류(ResNet)에서 여전히 강력한 기본기

현대 최적화 ② Adam 과 AdamW

파라미터마다 학습률을 «기울기 크기의 이동 평균»으로 자동 조절 (adaptive) + 모멘텀. AdamW 는 가중치 감쇠를 분리

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1-\beta_1)\mathbf{g}_t, \quad \mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1-\beta_2)\mathbf{g}_t^2, \quad \mathbf{W}_{t+1} = \mathbf{W}_t - \alpha\left(\frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \varepsilon} + \lambda \mathbf{W}_t\right)$$

1차 모멘트 m_t

- 기울기의 지수 이동 평균 (β_1 = 0.9)
- = 모멘텀 역할
- 초기 편향 보정 $\hat{m} = m/(1-\beta_1{}^t)$

2차 모멘트 v_t

- 기울기 제곱의 이동 평균 (β_2 = 0.999)
- √v 로 나누면 파라미터별 학습률 정규화
- 드물게 갱신되는 파라미터에 큰 스텝

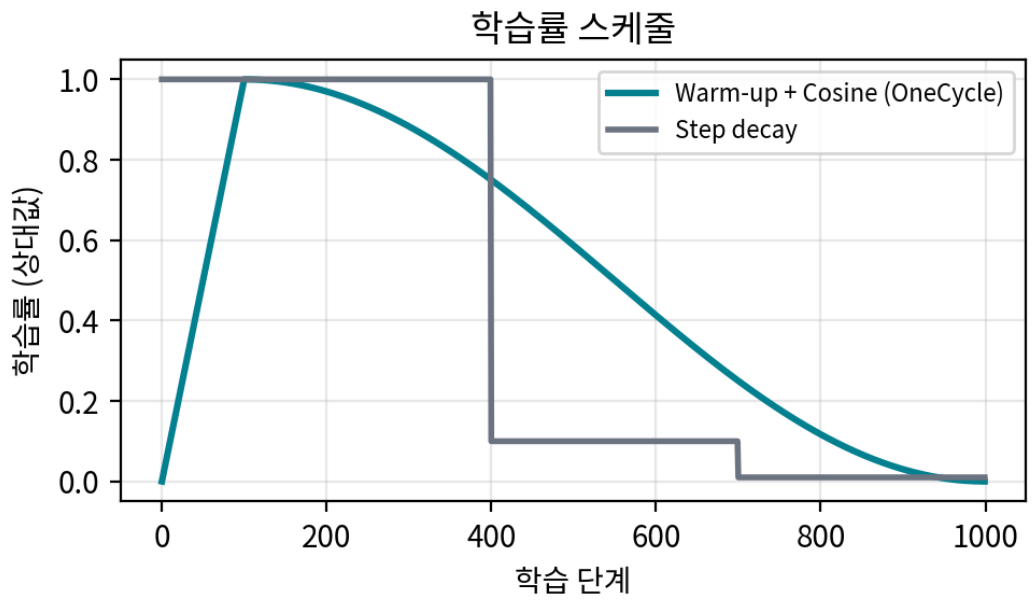
AdamW 의 λW_t 항

- L2 정규화를 기울기에 섞지 않고 «감쇠»로 분리 (Loshchilov & Hutter 2019)
- Adam+L2 는 적응 스케일에 왜곡됨
- 트랜스포머·LLM 의 사실상 표준

최적화기	학습률 기본값	권장 상황
SGD + momentum	0.01 ~ 0.1	CNN 이미지 분류, 일반화 중시
Adam	1e-3	빠른 프로토타이핑, RNN, GAN
AdamW	1e-4 ~ 3e-3 (+ weight_decay 0.01~0.1)	트랜스포머, 미세조정, 실습 2 기본

현대 최적화 ③ 학습률 스케줄 (Learning-Rate Schedule)

초반 warm-up 으로 안정, 후반 감쇠(annealing)로 정밀 수렴 — 고정 학습률보다 거의 항상 낫다




```
opt = torch.optim.AdamW(model.parameters(), lr=2e-3, weight_decay=1e-2)
sch = torch.optim.lr_scheduler.OneCycleLR(opt, max_lr=2e-3,
          epochs=EPOCHS, steps_per_epoch=len(train_loader))
for xb, yb in train_loader:
    loss = criterion(model(xb), yb); opt.zero_grad(); loss.backward()
    opt.step(); sch.step()          # ← 스텝마다 학습률 갱신
```

스케줄	PyTorch	특징
Step decay	StepLR / MultiStepLR	정해진 에폭마다 ×0.1 (고전 CNN)
Cosine annealing	CosineAnnealingLR	부드럽게 0 근처까지 감소 (현대 표준)
Warm-up + cosine	OneCycleLR, LambdaLR	초반 선형 증가 후 코사인 감소 (트랜스포머·LLM)
ReduceLROnPlateau	ReduceLROnPlateau	검증 손실 정체 시 감소 (적응형)

- 왜 warm-up 인가: 초기 가중치·Adam 의 v_t 추정이 불안정할 때 큰 학습률은 발산 → 처음 수백~수천 스텝은 작게
- 왜 감쇠인가: 미니배치 잡음 때문에 고정 학습률로는 최소점 주변을 «맴돈다» → 학습률을 줄여 정착
- **실습 2: OneCycleLR(max_lr=2e-3) — 5 에폭 만에 테스트 정확도 98.5%**
  - 매 step 마다 scheduler.step() 호출 (OneCycle) vs 매 epoch 호출 (StepLR·Cosine) — 문서 확인 필수

현대 학습 기법 ① 배치 정규화 · 레이어 정규화

각 층의 입력 분포를 평균 0 · 분산 1 로 재조정한 뒤 학습 가능한 γ, β 로 다시 스케일 — 학습 안정성과 속도의 핵심

$$\hat{z} = \frac{z - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \varepsilon}}, \qquad \tilde{z} = \gamma \hat{z} + \beta$$

- **효과**
  - 내부 공변량 이동 감소(원논문 주장) / 손실 지형을 매끄럽게 만들어 큰 학습률 허용 (Santurkar et al. 2018)
  - 초기화 민감도 ↓, 기울기 소실·폭주 완화, 약간의 정규화(regularization) 효과
- **배치 위치**
  - Linear → Norm → 활성화 (원논문) 또는 Norm → Linear (Pre-LN, 트랜스포머) — 실습 2 MLP 클래스는 전자
- 실습 2 실험 D: BatchNorm 추가 시 1 에폭 검증 정확도 95.7% → 96.3%

	BatchNorm (2015)	LayerNorm (2016)
통계 계산 축	미니배치 내 샘플들 (특징별)	샘플 하나의 특징들 (샘플별)
배치 크기 의존	있음 (작은 배치에서 불안정)	없음
학습/추론 차이	추론 시 이동 평균 μ, σ 사용 → model.eval() 필수	없음 (동일 계산)
주 사용처	CNN (ResNet 등)	트랜스포머·RNN·LLM (RMSNorm 변형)
PyTorch	nn.BatchNorm1d/2d	nn.LayerNorm, nn.RMSNorm

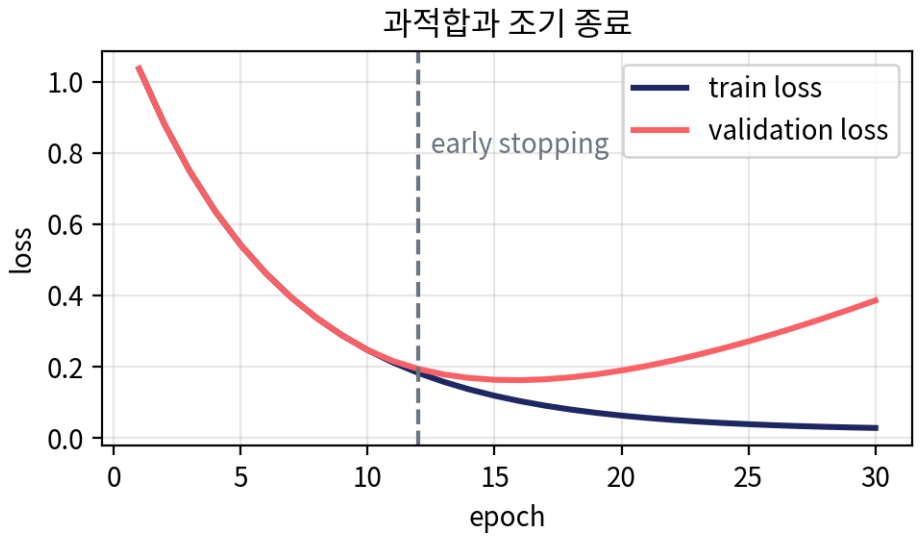
RMSNorm (Zhang & Sennrich 2019): 평균 빼기를 생략하고 RMS 로만 나눔 — LLaMA·Gemma 등 최신 LLM 의 기본. PyTorch 2.4+ nn.RMSNorm

주의: 정규화 층이 있으면 그 앞 Linear 의 바이어스는 불필요

Norm 이 평균을 빼므로 b 가 상쇄된다 → nn.Linear(..., bias=False) 로 파라미터 절약 (트랜스포머 구현에서 흔함)

현대 학습 기법 ② 과적합 제어 — 드롭아웃 · 가중치 감쇠 · 조기 종료

훈련 손실은 계속 내려가는데 검증 손실이 올라가기 시작하면 «외우기» 시작한 것



$$\tilde{h}_j = \frac{m_j}{1-p} h_j, \qquad m_j \sim \text{Bernoulli}(1-p)$$

조기 종료 (Early stopping)

- 검증 손실이 k 에폭(patience) 동안 개선되지 않으면 중단하고 «최고 검증 성능 시점»의 가중치 복원
- 가장 싸고 효과적인 정규화 — 실습 2 «생각해 볼 문제 3»

드롭아웃 (Srivastava et al. 2014)

- 학습 시 노드를 확률 p 로 0 으로 → 매 스텝 다른 «부분 망» 학습
- 추론 시 전체 사용 (학습 시 1/(1−p) 스케일로 기대값 보정 — inverted dropout)
- nn.Dropout(p); model.train()/eval() 전환 필수

가중치 감쇠 (L2)

- 손실에 $(\lambda/2)\|W\|^2$ 추가 ≡ 매 스텝 W 를 $(1-\eta\lambda)$ 배로 축소
- 큰 가중치 억제 → 부드러운 함수
- AdamW 의 weight_decay=0.01~0.1

그 밖의 정규화

- 데이터 증강 (RandomAffine 등): 데이터 자체를 늘림 — 가장 효과 큼
- 레이블 스무딩, 미니배치 잡음, 작은 모델·적은 에폭도 모두 정규화
- 현대 관점: 과매개변수(over-parameterized) 모델도 SGD 의 «암묵적 정규화» 로 잘 일반화 (이중 하강 현상, Belkin et al. 2019)

현대 학습 기법 ③ 기울기 클리핑 · 혼합 정밀도 · torch.compile

«안 터지게, 빠르게» — 대규모 학습의 실무 필수 요소 (실습 2 §4, §7)

기울기 클리핑

- $\|\nabla\|_2 > c$ 이면 $\nabla \leftarrow c \cdot \nabla / \|\nabla\|_2$ (방향 유지, 크기 제한)
- 폭주 방지, 학습 초기 안정화
- torch.nn.utils.clip_grad_norm_(params, 1.0) — 트랜스포머 표준값 1.0

혼합 정밀도 (AMP)

- 행렬곱은 float16/bfloat16, 누적·가중치는 float32
- 메모리 ½, 속도 2~3배 (텐서 코어)
- float16 은 GradScaler 로 언더플로 방지; bfloat16 은 불필요

torch.compile (2.0+)

- 파이썬 코드를 그래프로 캡처해 커널 융합 (TorchInductor)
- 첫 호출 컴파일 비용, 이후 1.3~2배 가속
- compiled_model = torch.compile(model)

```
scaler = torch.amp.GradScaler(enabled=(device.type == "cuda"))
for xb, yb in train_loader:
    with torch.autocast(device_type=device.type, dtype=torch.float16):    # 혼합 정밀도 순전파
        loss = criterion(compiled_model(xb), yb)
    optimizer.zero_grad(set_to_none=True)
    scaler.scale(loss).backward()                                         # 스케일된 역전파
    scaler.unscale_(optimizer); torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)   # 클리핑
    scaler.step(optimizer); scaler.update(); scheduler.step()
```

한 스텝의 표준 순서: 순전파(autocast) → 손실 → zero_grad → backward → (unscale) → 클리핑 → step → 스케줄러

모델의 성능 평가 지표 (Evaluation Metrics)

손실은 학습용, 지표는 보고용 — 문제 유형과 비용 구조에 맞는 지표를 고른다

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$$

- **회귀**
  - R^2 = 1 − SSE/SST: 평균 예측 대비 설명된 분산 비율 (1 이 최선, 0 이하면 평균보다 못함)
  - RMSE, MAE: y 와 같은 단위 → 해석 쉬움

	예측 Positive	예측 Negative
실제 Positive	TP	FN
실제 Negative	FP	TN

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}, \quad F_1 = \frac{2PR}{P + R}$$

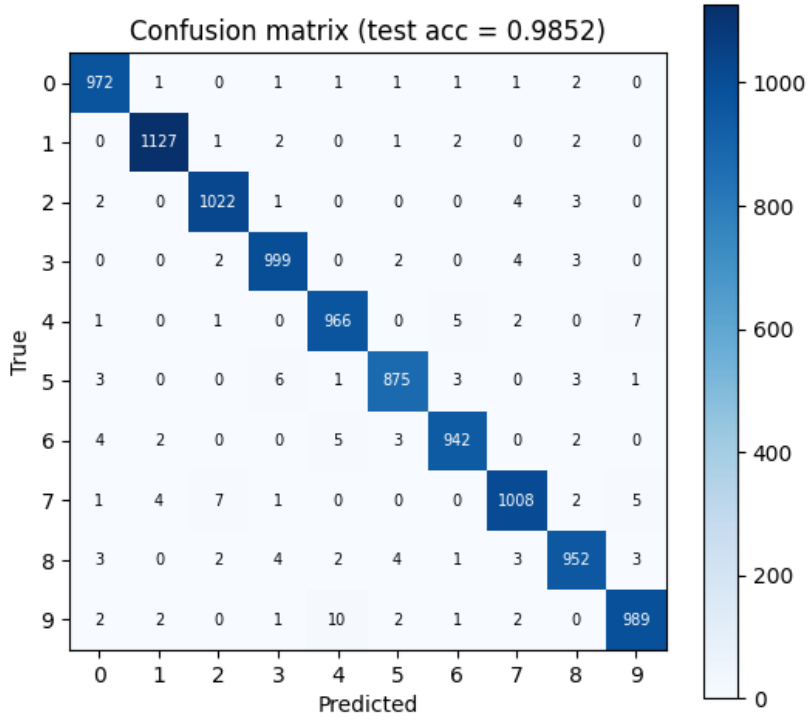
- **분류**
  - 정확도 = (TP+TN)/전체 — 클래스 균형일 때만 적절 (99% 음성인 데이터에서 «모두 음성» 도 99%)
  - 정밀도: 양성 예측 중 진짜 비율 (오경보 비용↑일 때), 재현율: 실제 양성 중 찾아낸 비율 (놓침 비용↑일 때 — 질병 선별)
  - F1 = 정밀도·재현율의 조화평균; 다중 클래스는 macro/micro 평균. 임계값 무관 지표: ROC-AUC, PR-AUC
  - 혼동행렬(C×C): 어떤 클래스끼리 헷갈리는지 → 오류 분석의 출발점 (다음 슬라이드)

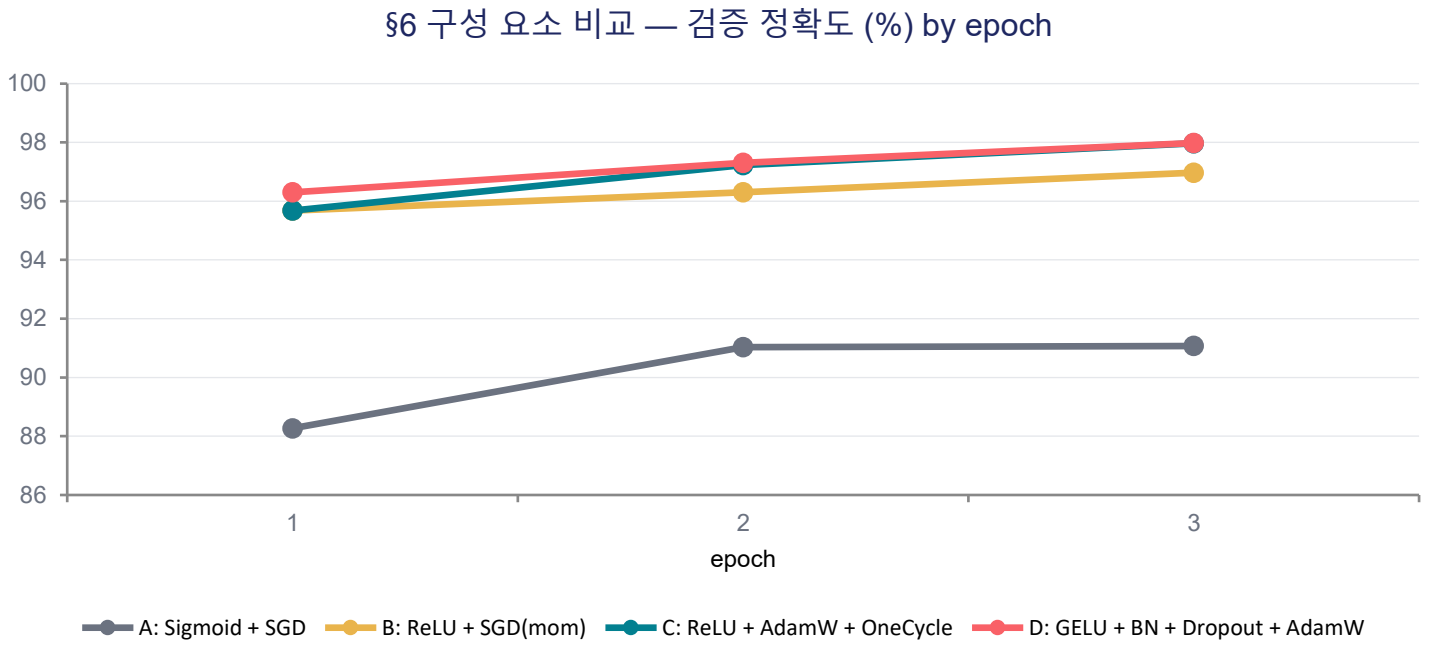
보정(calibration)과 확률

소프트맥스 확률 0.9 가 실제로 90% 맞는가? — 현대 신경망은 과신 경향 (Guo et al. 2017). 온도 스케일링, ECE 지표로 평가

실습 2 결과 — MNIST MLP (784-600-500-10) 학습·평가

5 에폭, AdamW + OneCycle, CPU 약 13초/에폭 · 테스트 정확도 98.52%, 오분류 148/10,000





- 혼동행렬(좌): 대각선이 지배적. 남은 오류는 4↔9, 7↔2, 4↔6, 3↔5 처럼 «모양이 비슷한 쌍»에 집중 — 클래스별 정확도 97.7% (8) ~ 99.3% (1)
- 실험 A→B: 활성화 함수(ReLU)와 모멘텀만으로 +6%p. B→C: AdamW + 스케줄 +1%p. C→D: BN + Dropout 은 초반 수렴을 빠르게 (1 에폭 +0.6%p)
- MLP 의 한계: 픽셀 위치 이동에 취약, 파라미터 77만 → 같은 예산의 CNN(10주차)은 99.3%+ 달성

PyTorch 학습 루프 — 이 강의의 모든 요소가 한 화면에 (실습 2 §4)

손실 · 최적화기 · 스케줄러 · 역전파 · 클리핑 · 평가 모드 전환

```
class MLP(nn.Module):
    def __init__(self, in_dim=784, h1=600, h2=500, n_classes=10, act=nn.ReLU, use_bn=False, dropout=0.0):
        super().__init__()
        layers = [nn.Flatten()]
        for din, dout in [(in_dim, h1), (h1, h2)]:
            layers += [nn.Linear(din, dout)] + ([nn.BatchNorm1d(dout)] if use_bn else []) + [act()] \
                    + ([nn.Dropout(dropout)] if dropout > 0 else [])
        layers.append(nn.Linear(h2, n_classes))          # 로짓 출력 (Softmax 없음)
        self.net = nn.Sequential(*layers)
        for m in self.net:                                # He 초기화
            if isinstance(m, nn.Linear): nn.init.kaiming_normal_(m.weight, nonlinearity="relu");
nn.init.zeros_(m.bias)
    def forward(self, x): return self.net(x)

criterion = nn.CrossEntropyLoss()                                          # 소프트맥스 + CCE
optimizer = torch.optim.AdamW(model.parameters(), lr=2e-3, weight_decay=1e-2)
scheduler = torch.optim.lr_scheduler.OneCycleLR(optimizer, max_lr=2e-3, epochs=5,
steps_per_epoch=len(train_loader))
for epoch in range(5):
    model.train()
    for xb, yb in train_loader:                                            # 미니배치 SGD
        loss = criterion(model(xb), yb)                                    # 순전파 + 손실
        optimizer.zero_grad(set_to_none=True); loss.backward()             # 오차 역전파
        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)            # 기울기 클리핑
        optimizer.step(); scheduler.step()                                 # 갱신 + 학습률 스케줄
    val_loss, val_acc = evaluate(model, val_loader)                        # model.eval() + no_grad
```

슬라이드 ↔ 코드

- 손실 함수 (7~11) → criterion
- 경사 하강·SGD (12~13) → optimizer / DataLoader
- 역전파 (14~18) → loss.backward()

현대 기법

- 모멘텀·Adam (20~21) → AdamW
- 스케줄 (22) → OneCycleLR
- BN·Dropout·클리핑 (23~25)

평가

- 지표 (26) → evaluate()
- 혼동행렬·오분류 사례 (27)
- 실습 §7: AMP + torch.compile

최신 동향: 학습 알고리즘 연구의 현재

역전파 + 적응형 최적화기의 틀은 40년째 유지되지만, 규모·효율·대안에 대한 연구가 활발하다

스케일링 법칙

- 손실이 파라미터 수·데이터·계산량의 거듭제곱으로 감소 (Kaplan 2020; Hoffmann/Chinchilla 2022)
- «얼마나 크게, 얼마나 오래 학습할까» 를 정량적으로 설계

새로운 최적화기

- Lion (2023): 부호 기반 갱신, 메모리 ½
- Muon (2024~25): 행렬 파라미터를 직교화해 갱신 — LLM 사전학습 효율 보고
- Shampoo/SOAP: 2차 정보 근사

메모리 효율 학습

- 8-bit Adam, ZeRO/FSDP 분산 최적화기 상태
- LoRA/QLoRA: 소수 파라미터만 학습 (미세조정)
- 그래디언트 체크포인팅: 활성값 재계산으로 메모리 ↓

역전파의 대안·보완 (연구 단계)

- Forward-Forward (Hinton 2022): 역전파 없이 층별 «좋음» 측정
- Direct Feedback Alignment, 국소 학습 규칙 — 생물학적 타당성·하드웨어 친화
- 아직 대규모 성능은 역전파에 못 미침 — 역전파를 정확히 이해하는 것이 여전히 기본

이론적 이해

- 손실 지형(loss landscape)의 평탄 최솟값과 일반화 (SAM 최적화기, Foret 2021)
- 이중 하강, 과매개변수 모델의 암묵적 정규화
- NTK(신경 접선 커널)·특징 학습 이론 — 왜 SGD 가 잘 작동하는가에 대한 답을 찾는 중

핵심 요약

6-2 다층 퍼셉트론의 학습

학습의 정식화

$W^* = \arg\min (1/N) \Sigma \ell(y_i, \hat{y}_i)$. 데이터 · 손실 · 최적화 세 요소, 훈련/검증/테스트 분리

손실 함수

회귀 MSE, 이진 BCE, 다중 CCE. 시그모이드/소프트맥스와 결합하면 $\partial L/\partial z = \hat{y} - y$ 로 단순

경사 하강법

$W \leftarrow W - \eta \nabla L$. 미니배치 SGD 가 표준, 학습률은 가장 중요한 하이퍼파라미터 + 스케줄

오차 역전파

연쇄법칙을 층 단위로: $\delta^{(k)} = (W^{(k)\top}\delta^{(k+1)}) \odot f'(z^{(k)})$, $\partial L/\partial W = \delta\, h^\top$. autograd 가 자동 수행

현대 학습 기법

He 초기화 · ReLU/GELU · BN/LN · 잔차 · 드롭아웃 · AdamW · warm-up+cosine · 클리핑 · AMP

평가

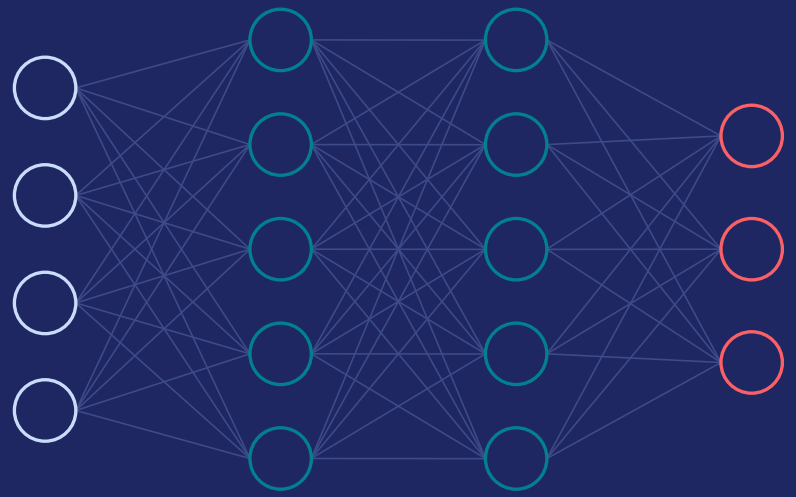
손실 ≠ 지표. 정확도·혼동행렬·정밀도/재현율/F1·R^2. MNIST MLP 98.5% — 10주차 CNN 으로

질문 및 정리

Q & A

오늘의 핵심 정리

- 학습 = 손실 최소화: 데이터 · 손실 함수 · 최적화
- MSE / BCE / CCE — 출력층 활성화와 짝을 이룬다
- 경사 하강법 + 미니배치 + 학습률 스케줄
- 역전파 = 연쇄법칙의 효율적 구현, autograd 로 검증
- 소실·폭주를 막는 현대 기법: 초기화·정규화·잔차·AdamW
- 검증 데이터로 감시, 테스트는 한 번 — 지표는 문제에 맞게



성명 : 최승호 · 이메일 : jcn99250@naver.com